The Automated Weather Dashboard System

The Scenario

You have just been hired as a Backend Developer for **ABC .**, a data analytics startup. Your first major project is to build an end-to-end Automated Weather Dashboard.

Your application must reach out to a public weather API, save the raw data into a structured local file system for archival, parse and insert that data into a relational database, and finally, expose this data and control the pipeline through a Flask web application.

You must utilize advanced Python concepts to ensure the application is modular, fault-tolerant, and secure.

Project Architecture & Instructions

Part 1: Project Structure & Directory Management (Modules & Directories)

To keep the codebase maintainable, you cannot write everything in one file.

1. Create a main project directory. Within it, use the `os` module to dynamically create an archive folder structure based on the current year and month (e.g., `./archives/2026/10/`).
2. Use `os.getcwd()` to print and log the absolute path of your script on startup to ensure your file paths are resolving correctly.
3. Separate your codebase into three distinct modules: `api_handler.py`, `db_manager.py`, and `app.py` (the main Flask file).
4. Ensure that you use specific imports across your files (e.g., `from db_manager import insert_data`).
5. Guarantee that your utility modules (`api_handler.py` and `db_manager.py`) **do not** execute any functional code if they are merely imported. Use the `if __name__ == '__main__':` block to isolate any local testing code within those files.

Part 2: Fetching Data & Handling Failures (HTTP & Exceptions)

In `api_handler.py`, write a function that reaches out to the open web to get data.

1. Use the `requests` library to make a `GET` request to a free, no-key API. *(Recommendation: Use*

`https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&current_weather=true`).
2. Check the HTTP status code of the response.
3. **Error Handling:** Define a custom exception class called `APIFetchError` that inherits from Python's base `Exception` class.
4. Wrap your HTTP request in a `try...except` block. If the network drops or the status code is not 200, explicitly `raise` your `APIFetchError`.
5. Use an `assert` statement to verify that the JSON response contains the specific key `"current_weather"` (or your chosen API's equivalent) before returning the parsed dictionary.
6. Use a `finally` block to print a message to the console indicating that the API connection attempt has finished, regardless of whether it succeeded or failed.

Part 3: Archival & Database Transactions (File I/O & Databases)

In `db_manager.py`, build the persistence layer. You may use SQLite (built-in) or MySQL.

1. **File I/O:** Write a function that takes the raw JSON string from Part 2 and writes it to a new `.txt` file inside the archive directory you created in Part 1.
2. **File Cleanup:** Write a small utility function that checks if an old file named `temp_cache.txt` exists in the root directory using the `os` module. If it does, safely `remove()` it before saving any new data.
3. **Database Setup:** Write a script/function to initialize your database and create a table named `weather_logs` with columns: `id` (Primary Key), `timestamp`, and `temperature`.
4. **Transactions:** Create an `insert_weather_data(temp, time)` function.
5. You **must** use parameterized queries (using `?` for SQLite or `%s` for MySQL) in your `INSERT` statement to prevent SQL injection vulnerabilities.
6. Wrap the insert operation in a `try...except` block. If an error occurs during insertion, execute a `rollback()` on the database connection. If successful, execute a `commit()`.

Part 4: Public Web Interface & Pipeline Control (Flask)

In `app.py`, tie everything together and expose it via the web.

1. Initialize a Flask application.
2. Define a home route (`/`) that responds to `GET` requests. It should use `render_template` to serve a simple HTML page displaying a welcome message.
3. Create a REST API endpoint at `/api/v1/weather` that queries your database, fetches all records from the `weather_logs` table, and returns them to the browser as a cleanly formatted JSON response using Flask's `jsonify`.
4. **The Trigger:** Create a route at `/api/v1/update` that accepts `POST` requests. This route acts as your application's control button. When triggered, it should:

- Call your function from `api_handler.py` to get live data.
- Call your function from `db_manager.py` to save the data to the file and database.
- Catch the `APIFetchError` if it occurs, returning a `{"status": "error"}` JSON response with a 500 status code.
- Return a `{"status": "success"}` JSON response with a 200 status code if everything works.